Contents

Linux — Interview Questions & Answers **1**
Basic / Mid-Level 1
Advanced / Senior Level 2
Scripting / Automation Questions 3

Linux — Interview Questions & Answers

Basic / Mid-Level

Q1: What is the difference between a hard link and a soft link? > **Hard link:** Another directory entry pointing to the same inode. Cannot span filesystems. Deleting original doesn't remove data until all hard links deleted. > **Soft link (symlink):** Points to a path/name. Can span filesystems. Breaks if target is deleted. > `ln file hardlink` vs `ln -s file softlink`

Q2: Explain the Linux boot process. > 1. BIOS/UEFI POST → finds bootable device > 2. MBR/GPT → loads bootloader (GRUB2) > 3. GRUB2 → loads kernel image + initramfs into memory > 4. Kernel initializes hardware, mounts initramfs (temporary root FS) > 5. Kernel mounts real root FS, starts `/sbin/init` (systemd on modern distros) > 6. Systemd reads `/etc/systemd/system/`, starts default target (multi-user.target or graphical.target) > 7. Login prompt

Q3: What is the difference between `ps aux` and `ps -ef`? > Both show all processes. aux is BSD style (shows %CPU, %MEM, VSZ, RSS). `-ef` is POSIX style. `ps aux` additionally shows non-tty processes.

Q4: How do you find which process is listening on port 8080?

```
ss -tlnp | grep :8080
lsof -i :8080
fuser 8080/tcp
```

Q5: What is a zombie process? How do you remove it? > A process that has finished execution but its parent hasn't called `wait()` to collect its exit status. It holds a PID and entry in the process table. Kill the **parent** process to clean up zombies (they'll be re-parented to init which will reap them).

Q6: What is `nice` and `renice`? > `nice` sets the priority when starting a process (-20 = highest, 19 = lowest, default 0). > `renice` changes the nice value of a running process.

```
nice -n 10 make -j4
renice -n -5 -p 1234    # requires root for negative values
```

Q7: Explain `cron` syntax.

```
*  *  *  *  *  command
│  │  │  │  └─ Day of week (0-7, Sun=0)
│  │  │  └──── Month (1-12)
│  │  └─────── Day of month (1-31)
│  └────────── Hour (0-23)
└───────────── Minute (0-59)

0 2 * * 0 /scripts/backup.sh    # Every Sunday at 2am
*/15 * * * * /scripts/health.sh # Every 15 minutes
@reboot /scripts/startup.sh     # At boot
```

Q8: What is the difference between > and >>? > > overwrites the file. >> appends to the file. > 2> redirects stderr. 2>&1 redirects stderr to stdout. &> redirects both.

Q9: How do you check disk usage?

```
df -h           # filesystem usage
du -sh /var/log # directory size
du -h --max-depth=1 /var  # one level deep
ncdu /var       # interactive (if installed)
```

Q10: What are environment variables and how are they set?

```
export VAR=value          # current shell + children
echo $VAR
env                       # view all
unset VAR                 # remove
printenv HOME
# Persistent: add export to ~/.bashrc, ~/.bash_profile, /etc/environment
```

Advanced / Senior Level

Q11: Explain Linux namespaces and how Docker uses them. > Linux namespaces provide process isolation by limiting what a process can see. Docker uses: > - **PID namespace:** Container processes see only their own PID space (PID 1 in container ≠ PID 1 on host) > - **NET namespace:** Container gets its own network stack (eth0, lo, routing table) > - **MNT namespace:** Container sees only its own mount tree > - **UTS namespace:** Container can have its own hostname > - **IPC namespace:** Isolated IPC resources > - **USER namespace:** (optional) Map container root to non-root host user

Q12: What are cgroups and how do they relate to containers? > Control groups limit and account for resource usage of process groups. Docker uses cgroups to: > - Limit CPU (`--cpus=2`), memory (`--memory=512m`) > - Limit disk I/O bandwidth > - Track resource consumption per container > `docker run --cpus=1.5 --memory=1g nginx`

Q13: How does `iptables` work? How does Docker manage iptables? > iptables works with kernel netfilter hooks. Packets go through chains (INPUT, OUTPUT, FORWARD, PREROUTING, POSTROUTING). Docker: > - Creates DOCKER chain in FORWARD and nat tables > - Adds DNAT rules in PREROUTING for port mappings > - Sets FORWARD default to DROP but allows docker network traffic > - Sets masquerade (SNAT) in POSTROUTING for outbound container traffic

Q14: What is the difference between soft and hard limits (`ulimit`)? > **Soft limit:** Current effective limit; can be raised up to hard limit by the user. > **Hard limit:** Maximum that unprivileged users can set; only root can raise it.

```
ulimit -a              # show all limits
ulimit -n 65535        # set open files (soft)
ulimit -Hn 65535       # set open files (hard, root only)
# Persistent: /etc/security/limits.conf
* soft nofile 65535
* hard nofile 65535
```

Q15: Explain `strace`. When do you use it? > `strace` traces system calls made by a process. Used for debugging crashes, permission issues, file not found errors.

```
strace -p PID                    # attach to running process
strace -e trace=open,read ls     # filter specific syscalls
strace -o /tmp/trace.log -f cmd  # follow forks, log to file
strace -c program                # summary of syscalls + timing
```

Q16: A server is running out of memory but `free -h` shows available memory. What could be wrong? > - **Swap exhaustion:** Check `swapon -s` or `vmstat` > - **OOM killer invoked:** Check `dmesg | grep -i oom` > - **Memory leaks in application:** Check `pmap -x PID | tail -1` > - **File descriptor exhaustion:** Check `ulimit -n` and `/proc/PID/fd` > - **Kernel memory (slab):** Check `slabtop` or `/proc/slabinfo` > - **Dirty page writeback:** `cat /proc/meminfo | grep Dirty`

Q17: How do you debug a "Too many open files" error?

```
# Check limits
ulimit -n
cat /proc/sys/fs/file-max    # system-wide max

# Check current usage
lsof | wc -l                 # total open files
lsof -p PID | wc -l          # per process
```

```
ls /proc/PID/fd | wc -l      # directly

# Check system-wide
cat /proc/sys/fs/file-nr     # used / free / max

# Fix
ulimit -n 65535              # temporarily
echo "* soft nofile 65535" >> /etc/security/limits.conf
echo "* hard nofile 65535" >> /etc/security/limits.conf
```

Q18: Explain the difference between /proc and /sys. > **/proc:** Process information + kernel interfaces. Contains a directory per PID. Also has system info: /proc/cpuinfo, /proc/meminfo, /proc/net/. > **/sys (sysfs):** Structured view of kernel devices and drivers. Used to modify kernel parameters at runtime. More organized than /proc.

Q19: What is a segmentation fault? How do you debug it? > SIGSEGV — process tried to access memory it doesn't have permission to (null pointer, stack overflow, buffer overflow).

```
# Enable core dumps
ulimit -c unlimited
echo "/tmp/core.%e.%p" > /proc/sys/kernel/core_pattern

# After crash
gdb /usr/bin/myapp /tmp/core.myapp.1234
(gdb) bt       # backtrace
(gdb) info registers

# Other tools
valgrind --leak-check=full ./myapp   # memory error detector
AddressSanitizer (compile with -fsanitize=address)
```

Q20: How do you optimize Linux for running Kubernetes?

```
# Disable swap
swapoff -a
# Remove swap from /etc/fstab

# Load required modules
modprobe br_netfilter overlay

# Kernel params
cat > /etc/sysctl.d/k8s.conf << EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables  = 1
net.ipv4.ip_forward                 = 1
vm.overcommit_memory                = 1
vm.panic_on_oom                     = 0
kernel.panic                        = 10
kernel.panic_on_oops                = 1
fs.inotify.max_user_watches         = 524288
fs.inotify.max_user_instances       = 512
EOF
sysctl --system
```

Scripting / Automation Questions

Q21: Write a script to find and archive log files older than 30 days.

```
#!/usr/bin/env bash
set -euo pipefail
```

```
LOG_DIR="/var/log/app"
ARCHIVE_DIR="/backup/logs"
RETENTION_DAYS=30

find "$LOG_DIR" -name "*.log" -mtime +"$RETENTION_DAYS" -print0 | \
  tar -czvf "$ARCHIVE_DIR/archive-$(date +%Y%m%d).tar.gz" --null -T - && \
  find "$LOG_DIR" -name "*.log" -mtime +"$RETENTION_DAYS" -delete

echo "Archived logs older than $RETENTION_DAYS days"
```

Q22: How would you monitor a process and restart it if it crashes?

```
#!/usr/bin/env bash
PROCESS="myapp"
while true; do
    if ! pgrep -x "$PROCESS" > /dev/null; then
        echo "$(date): $PROCESS not running, restarting..." | \
          tee -a /var/log/watchdog.log
        /usr/local/bin/$PROCESS &
    fi
    sleep 30
done
# Better: Use systemd with Restart=always or supervisord
```

Q23: Write a script to check if all required ports are open before starting a service.

```
#!/usr/bin/env bash
check_port() {
    local host="$1" port="$2"
    timeout 2 bash -c "echo >/dev/tcp/$host/$port" 2>/dev/null
}

DEPS=("db:5432" "redis:6379" "kafka:9092")
for dep in "${DEPS[@]}"; do
    host="${dep%:*}"
    port="${dep##*:}"
    if check_port "$host" "$port"; then
        echo "✓ $host:$port reachable"
    else
        echo "✗ $host:$port UNREACHABLE — aborting"
        exit 1
    fi
done
echo "All dependencies ready. Starting service."
```